



Final Report

COMPANY NAME: 6 ANGRY MEN

Long-Range Camera-Based Drone Tracking System with Laser Alignment

31/05/2026

PRESENTED BY:

Ahmet Kerem Çolakoğlu, Ahmet Burak Taylan,

Hamza Solmaz, Hüseyin Sinan Demir,

Ömer Taha Serin, Muhammed Furkan İçten

Project Start Date: 31.10.2025

Project End Date: 13.05.2026

Table of Contents

Executive Summary.....	3
Introduction.....	3
Design Description.....	4
Results and Analyses of Performance Tests.....	7
List of Deliverables.....	8
Hardware Deliverables.....	8
Software Deliverables.....	8
Validation Deliverables.....	8
Budget.....	8
Discussions.....	10
Safety Issues and Precautions.....	10
Potential Applications and Societal Impact.....	10
Environmental Impact.....	10
Conclusion.....	11
Appendix I.....	12
Appendix II.....	18

Executive Summary

This report documents the design, development, and testing of a Long-Range Camera-Based Drone Tracking System with Laser Alignment, built by the "6 Angry Men" team between October 2025 and June 2026.

The system was developed as a cost-effective alternative to radar-based UAV monitoring. It combines a YOLO-based detection pipeline, object tracking, PID-based visual servoing, and a motorized pan-tilt platform — all running on a Raspberry Pi 5. A laser module co-aligned with the camera axis ensures that centering the target in frame automatically directs the laser at the drone, with no separate aiming logic required.

Outdoor testing at the METU-EEE car park confirmed the core system capabilities: near-zero false positives against environmental distractors (birds, pedestrians, foliage), stable tracking of drones flying up to 4 m/s across horizontal and vertical axes, and confirmed laser-to-target alignment at 10, 20, and 30 meters. The main shortfall identified was detection latency — average initial acquisition time was 7-10 seconds, well above the 3-second target — and detection reliability degraded significantly against cluttered backgrounds.

Overall, the system meets its primary objectives at low cost and complexity. The detection latency and background sensitivity remain the key areas for future improvement.

Introduction

The rapid increase in the use of unmanned aerial vehicles (UAVs) has created a growing demand for systems capable of detecting, tracking, and monitoring aerial targets in real time. Such systems are required in a wide range of applications including security surveillance, critical infrastructure protection, research activities, and airspace monitoring. While modern radar-based solutions provide effective target detection capabilities, they are often associated with high costs, complex deployment requirements, and significant infrastructure needs. As a result, camera-based tracking systems have emerged as an attractive alternative due to their lower cost, flexibility, and ease of implementation.

This project presents the design, implementation, and validation of a Long-Range Camera-Based Drone Tracking System with Laser Alignment. The primary objective of the system is to detect a drone-sized target at long distances, continuously track its motion using computer vision algorithms, and maintain a visible laser pointer aligned with the target throughout the tracking process. The project combines multiple engineering disciplines, including computer vision, embedded systems, control engineering, electronics, and mechanical design, into a unified mechatronic platform.

The developed system operates as a closed-loop visual servoing system. Image frames acquired by a camera are processed on a Raspberry Pi 5, where a YOLO-based object detection algorithm identifies the target and determines its position within the image plane. The resulting positional information is then converted into control commands that drive a

pan-tilt mechanism equipped with NEMA 17 stepper motors and TMC2209 motor drivers. Since the laser module is mechanically aligned with the camera axis, maintaining the target at the center of the image frame simultaneously aligns the laser beam with the detected drone.

Throughout the project, particular emphasis was placed on achieving reliable real-time performance on embedded hardware while maintaining sufficient detection accuracy and tracking stability. Various subsystem-level and integrated tests were conducted to evaluate the perception, control, and actuation capabilities of the platform. These experiments demonstrated that the system is capable of detecting and tracking drone targets under different operating conditions while maintaining acceptable responsiveness and positioning accuracy.

The final system successfully integrates vision-based target detection, object tracking, motion control, and laser alignment into a single platform. The results obtained throughout the development process confirm the feasibility of the proposed approach and establish a solid foundation for future improvements such as multi-target tracking, advanced estimation algorithms, and autonomous search strategies.

Design Description

The developed system is a ground-based electro-mechanical tracking platform designed to detect, track, and visually mark aerial targets using a camera-laser assembly mounted on a motorized pan-tilt mechanism. The design follows a modular architecture consisting of four primary layers: the sensing layer, the perception layer, the command and control layer, and the actuation layer. This modular structure simplifies system integration, facilitates subsystem-level testing, and allows future upgrades without major architectural modifications.

The sensing layer consists of a Raspberry Pi Global Shutter Camera equipped with a telephoto lens optimized for long-range target observation. The camera continuously acquires image frames and provides visual information to the processing unit. The use of a global shutter sensor minimizes motion distortion and improves the reliability of target localization during dynamic tracking scenarios.

The perception layer is implemented on a Raspberry Pi 5 and serves as the computational core of the system. A YOLO-based object detection model processes incoming image frames and identifies drone targets within the field of view. For every detected target, the system generates a bounding box, confidence score, and image-plane coordinates. To improve temporal consistency and reduce detection noise, an object tracking algorithm is employed between consecutive detections. Following comparative testing of multiple OpenCV tracking methods, the CSRT tracker was selected due to its superior stability and resistance to tracking drift. The combination of detection and tracking enables the system to maintain continuous target awareness even under challenging motion conditions.

The command and control subsystem acts as the interface between the perception layer and the mechanical actuation stage. The center coordinates of the detected target are continuously compared with the center of the camera frame to calculate horizontal and vertical tracking errors. These errors are used as feedback signals for the control algorithm. Rather than performing computationally expensive three-dimensional reconstruction, the system utilizes a pixel-based visual servoing approach. This strategy significantly reduces processing requirements while providing sufficient accuracy for the intended tracking task.

To improve motion stability and tracking precision, a PID-based control algorithm is integrated into the command generation process. The controller continuously evaluates the image-plane error and generates corrective motion commands for both pan and tilt axes. By using visual feedback directly obtained from the camera stream, the system eliminates the need for external position sensors or mechanical encoders. This design choice simplifies the overall architecture while maintaining closed-loop control capability.

The actuation layer consists of two NEMA 17 stepper motors driven by TMC2209 motor drivers. Compared to the servo-based prototype evaluated during earlier development stages, the final stepper-motor-based architecture provides higher positioning accuracy, smoother movement, and improved repeatability. The TMC2209 drivers support microstepping operation, allowing the system to perform fine angular corrections required for long-range target tracking.

A critical mechanical design feature of the system is the permanent alignment between the camera optical axis and the laser module. This configuration eliminates the need for separate laser aiming calculations. Whenever the target is centered within the camera image, the laser is automatically directed toward the same location. This significantly reduces computational complexity while improving system reliability.

The final software architecture integrates image acquisition, object detection, object tracking, command generation, motor control, and system monitoring within a unified framework. During operation, the system continuously cycles through image acquisition, target localization, error calculation, motion correction, and visual feedback generation. This closed-loop operation enables real-time target tracking while maintaining laser alignment under dynamic conditions.

Overall, the final design successfully combines computer vision, embedded processing, control theory, and precision actuation into a compact and cost-effective tracking platform. Experimental testing confirmed that the selected architecture satisfies the primary project requirements regarding detection capability, tracking stability, positioning accuracy, and real-time operation.

The flowchart of the system architecture and complete product is given in Figure 1,2 respectively.

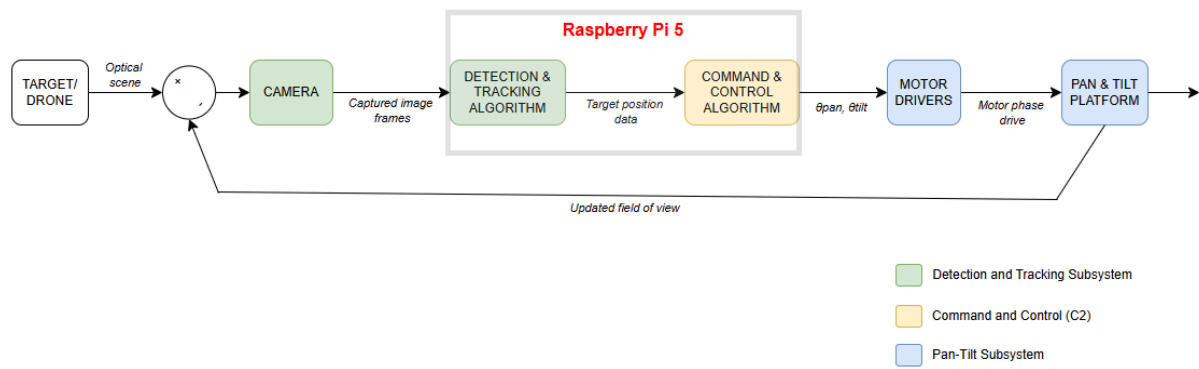


Figure 1. System architecture of the drone tracking project

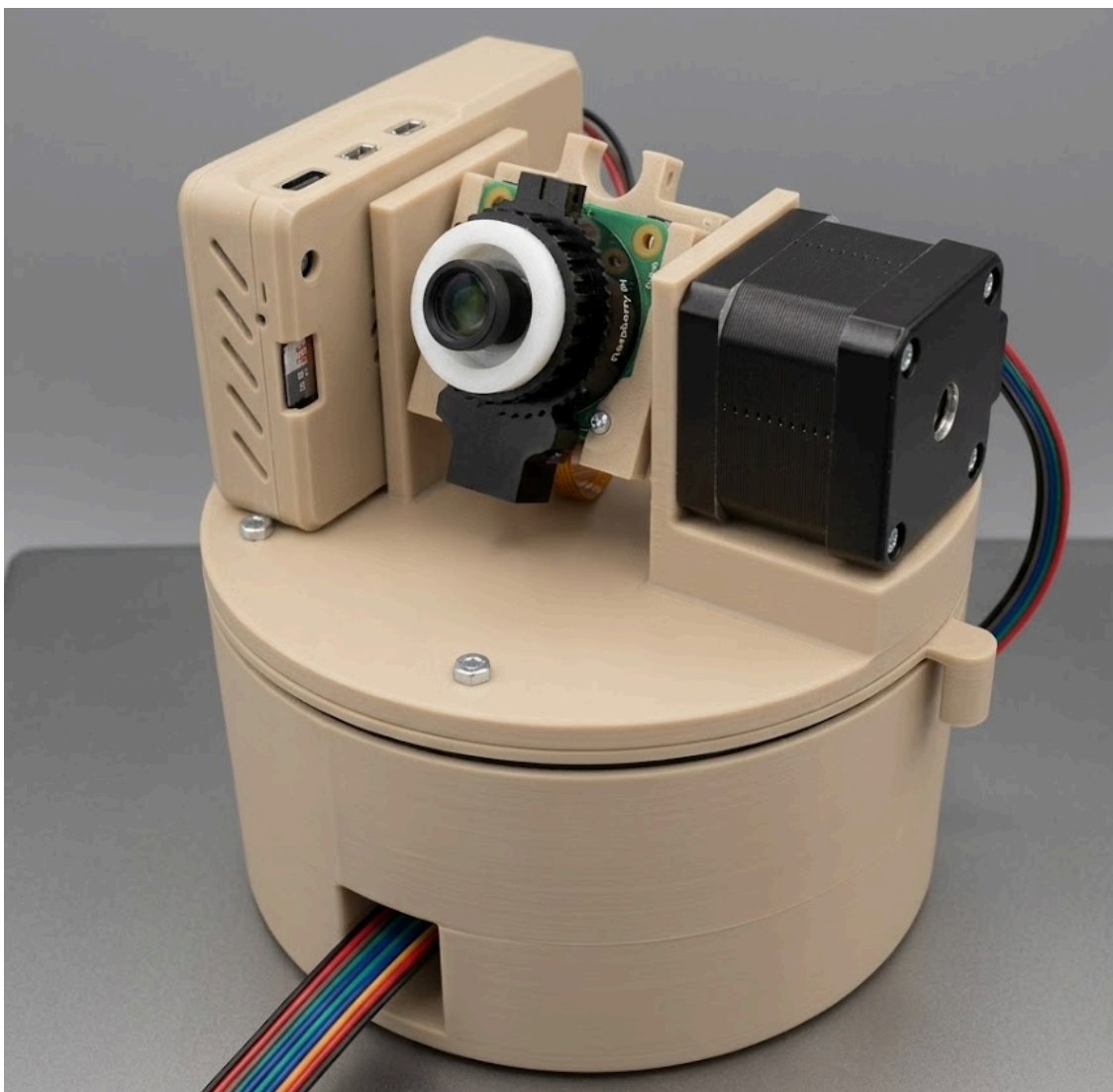


Figure 2. Finished product

Results and Analyses of Performance Tests

To validate the operational capabilities and performance boundaries of the developed drone tracking system, a comprehensive outdoor demo test was conducted at the METU-EEE Department car park environment. The experimental verification was structured across distinct phases, evaluating the system's computer vision pipeline, its real-time hardware response over multiple axes, and physical targeting accuracy.

The initial phase focused on assessing the reliability of the system's computer vision architecture in minimizing false positives within a dynamic environment. During the Non-Drone Object Test, the tracking system was powered on and exposed to a field of view containing active, non-target environmental elements such as moving tree branches, birds, and pedestrian traffic. The detection pipeline successfully ignored these environmental anomalies, generating zero false positives. This demonstrates that the feature extraction algorithms possess the necessary discrimination capabilities to avoid triggering unintended tracking sequences in clear, non-cluttered conditions.

Following false-positive verification, the detection latency was benchmarked under open-sky conditions. The test procedure involved positioning the drone at a range of approximately 35 meters and measuring the time elapsed from system activation to a stable visual lock while varying the drone's flight speed. While the system achieved successful target initialization across all trials, it did not satisfy the strict 3-second design requirement. The initial detection time varied based on the drone's spatial positioning, resulting in a calculated average latency of 7-10 seconds. This temporal delay represents a known baseline parameter of the current initialization pipeline under open-sky conditions. Conversely, when the drone was flown in front of complex backgrounds, such as structured buildings and dense trees, the detection pipeline was unable to differentiate the target from the high-clutter environment. Consequently, successful detection and target initialization remained limited to open-sky scenarios, establishing a distinct boundary for the system's operational background conditions.

The hardware tracking and servo-loop responsiveness were analyzed under active flight paths to measure dynamic stability. The single-axis tracking and dual-axis tracking evaluations were executed by flying the target drone along controlled horizontal, vertical, and diagonal trajectories within the system's 180-degree azimuth field of view. Throughout these maneuvers, the prototype demonstrated robust path-following capabilities. The system maintained a continuous target lock and smooth pan-and-tilt motor coordination without losing tracking continuity, provided the drone's velocity remained below 4 m/s in all directional axes. This successfully establishes the system's dynamic tracking envelope, confirming reliable multi-axis synchronization up to this velocity threshold.

Finally, the physical targeting precision and hardware alignment were verified via a ground truth assessment. The drone was stabilized at various operational ranges (10, 20, and 30 meters) while the system maintained an active tracking lock. Alignment accuracy was evaluated by inspecting the live camera feed displayed on the graphical user interface (GUI). The integrated laser module's projection appeared centered on the camera output GUI. This successful visual alignment confirms that the spatial calibration between the camera sensor

and the targeting laser is precise, ensuring exact correlation between the software's coordinate estimations and physical targeting vectors within the current hardware assembly.

List of Deliverables

Hardware Deliverables

- Fully assembled pan-tilt tracking platform
- Raspberry Pi 5 processing unit
- Raspberry Pi Global Shutter Camera
- 5MP M12 Lens
- Laser
- 17HS4401 Nema 17 Step Motor
- TMC2209 motor driver modules
- Power distribution and power supply system

Software Deliverables

- YOLO-based drone detection software
- CSRT-based target tracking software
- PID control implementation
- Pan-tilt motor control software
- Search and recovery mode algorithms
- System integration software
- Graphical User Interface (GUI)

Validation Deliverables

- Detection performance test results
- Tracking performance test results
- Pan-tilt positioning accuracy test results
- Demonstration videos
- User Manual

Budget

We completed the project while keeping the total cost below the specified budget limit. The products used in the project, along with their quantities and prices, are presented in the budget Table 1.

Table 1. Detailed Cost Table

Product	Count	Price (TL)	Price (Dollar)	Purchase Date	Dolar/TL when purchasing
Raspberry Pi 5 - 8GB	1	4.582,44	107,95	03.12.2025	42,45
Telefoto M12 Lens - 5 MP (25 mm, 1/2")	1	892,08	21,01	03.12.2025	42,45
Raspberry Pi Global Shutter Camera	1	3.675,00	86,57	03.12.2025	42,45
17HS4401S Nema 17 Step Motor	2	603,06	13,60	23.03.2026	44,33
TMC2209 Step Motor Sürücü	2	1122,88	25,33	23.03.2026	44,33
Active Cooler for RPi5	1	342,57	7,73	23.03.2026	44,33
Lazer	1	140,4	3,17		44,33
3D Print Parts	300 gr	210	4,74		44,33
Total		11.568,43	270,10		
Budget			300,00		
Remaining			29,90		

In addition, the Total Cost table includes spare parts and the revised 3D-printed component costs that were redesigned and reproduced throughout the development process. The complete breakdown of these expenditures is also provided in Table 2.

Table 2. Total Cost Table

Product	Count	Price (TL)	Price (Dollar)	Dolar/TL when purchasing
Materials (with extra material costs)		12763,64	287,92	44,33
Engineering Cost	300 hours	150000,00	3383,71	44,33
Test Flights	10	10000,00	225,58	44,33
Logistic Cost	5	500,00	11,28	44,33
Total		173263,64	3908,50	

Discussions

Safety Issues and Precautions

The developed system involves several safety considerations due to the use of moving mechanical components, electrical power systems, and a visible laser module. Potential hazards include unintended motor movements, mechanical collisions, overheating of electronic components, and accidental exposure to laser radiation. To minimize these risks, the system was designed with mechanical motion limits, controlled path via mechanical switches motor acceleration, current-limiting motor drivers, and dedicated power supplies for actuation subsystems. In addition, testing procedures were conducted in controlled environments, and the laser was operated only under supervised conditions. These precautions significantly reduced operational risks while ensuring reliable system performance.

Potential Applications and Societal Impact

The proposed drone tracking system has the potential to be utilized in a wide variety of civilian, industrial, and security-related applications. Possible use cases include perimeter surveillance, critical infrastructure monitoring, search and rescue operations, wildlife observation, and research activities involving unmanned aerial vehicles. The computer vision algorithms, tracking methods, and control strategies developed during the project may also be adapted to other autonomous monitoring systems. Widespread adoption of such technologies could increase operational efficiency, and reduce the need for continuous human supervision in monitoring tasks.

Environmental Impact

The environmental impact of the developed system is relatively limited due to its modest power consumption and compact hardware architecture. The platform does not generate direct emissions during operation and primarily relies on commercially available electronic components. However, large-scale deployment of similar systems may contribute to electronic waste generation at the end of product life cycles. Proper recycling of electronic components, responsible battery disposal practices, and the use of energy-efficient hardware can help minimize these environmental effects. Overall, the system represents a low-impact technological solution when operated and disposed of responsibly.

Conclusion

This report has presented the design, implementation, and experimental validation of a low-cost, ground-based Long-Range Camera-Based Drone Tracking System with Laser Alignment, successfully developed by the "6 Angry Men" team.

The project achieved its primary engineering objective: creating a modular mechatronic platform capable of real-time target tracking and physical laser alignment without the high infrastructure requirements and financial costs associated with traditional radar systems. By leveraging a unified hardware-software architecture—comprising a Raspberry Pi 5 computational core, a YOLO-based detection and CSRT tracking pipeline, and a precise NEMA 17 stepper-motor-driven pan-tilt mechanism—the platform realized a closed-loop visual servoing strategy. The physical implementation proved that co-aligning the optical axis of the global shutter camera with the laser module effectively eliminates complex spatial coordinate transformations, significantly reducing computational overhead while guaranteeing reliable target marking.

Experimental evaluations conducted in outdoor conditions verified that the platform demonstrates excellent stability under a clear open-sky environment, registering zero false positives against dynamic environmental distractors such as birds and pedestrians. The mechanical system and PID control loops showed robust coordination, maintaining continuous target locks with a width of 22 cm drone from a distance of at least 30 meters and tracking target speeds up to 4 m/s. Furthermore, the final implementation stayed successfully within the assigned financial constraints, demonstrating a highly cost-efficient product design with a total finalized expenditure of \$270.10 (well below the \$300.00 budget threshold).

Despite these successes, systematic testing exposed clear performance boundaries that establish areas for future development. The primary technical shortfall is the system's detection latency, which exhibited an average initialization time of 7-10 seconds falling short of the original 3-second design target. Additionally, the perception pipeline suffers from background sensitivity, restricting successful drone detection strictly to open-sky conditions and failing to differentiate targets against complex, cluttered backdrops like buildings or trees.

In conclusion, the developed prototype establishes a highly viable, robust, and cost-effective foundation for vision-based UAV monitoring. While the mechatronic actuation and spatial alignment function flawlessly, future iterations should focus heavily on upgrading the perception pipeline. Implementing more advanced, background-invariant deep learning architectures, migrating to more powerful embedded edge-AI hardware, or incorporating multi-target prediction and recovery algorithms will be essential steps to transition this system into a fully autonomous, commercially deployable security surveillance product.

Appendix I

Detection Code

```
9     @dataclass
10     class Detection:
11         class_id: int
12         class_name: str
13         confidence: float
14         bbox_xyxy: Tuple[int, int, int, int] # (x1, y1, x2, y2)
15
16         @property
17         def bbox_xywh(self) -> Tuple[int, int, int, int]:
18             x1, y1, x2, y2 = self.bbox_xyxy
19             return (x1, y1, x2 - x1, y2 - y1)
20
```

```
22     class YoloDetector:
23         def __init__(self, model_path: str, conf_threshold: float = 0.5):
24             self.model = YOLO(model_path, task="detect")
25             self.labels = self.model.names
26             self.conf_threshold = conf_threshold
27
28         def detect(
29             self,
30             frame,
31             target_class_name: Optional[str] = None
32         ) -> List[Detection]:
33             results = self.model(frame, verbose=False)
34             boxes = results[0].boxes
35
36             detections: List[Detection] = []
37
38             for i in range(len(boxes)):
39                 conf = float(boxes[i].conf.item())
40                 if conf < self.conf_threshold:
41                     continue
42
43                 class_id = int(boxes[i].cls.item())
44                 class_name = self.labels[class_id]
45
46                 if target_class_name is not None and class_name != target_class_name:
47                     continue
48
49                 xyxy = boxes[i].xyxy.cpu().numpy().squeeze().astype(int)
50                 x1, y1, x2, y2 = map(int, xyxy)
51
52                 detections.append(
53                     Detection(
54                         class_id=class_id,
55                         class_name=class_name,
56                         confidence=conf,
57                         bbox_xyxy=(x1, y1, x2, y2),
58                     )
59                 )
60
61             return detections
```

```

63  ✓    def get_best_detection(
64        self,
65        frame,
66        target_class_name: Optional[str] = None
67    ) -> Optional[Detection]:
68        detections = self.detect(frame, target_class_name=target_class_name)
69        if not detections:
70            return None
71        return max(detections, key=lambda d: d.confidence)
72
73    @staticmethod
74  ✓    def draw_detections(frame, detections: List[Detection]):
75        for det in detections:
76            x1, y1, x2, y2 = det.bbox_xyxy
77            label = f"{det.class_name}: {det.confidence:.2f}"
78
79            cv2.rectangle(frame, (x1, y1), (x2, y2), (0, 255, 0), 2)
80
81            (tw, th), baseline = cv2.getTextSize(
82                label, cv2.FONT_HERSHEY_SIMPLEX, 0.5, 1
83            )
84            y_text = max(y1, th + 10)
85            cv2.rectangle(
86                frame,
87                (x1, y_text - th - 10),
88                (x1 + tw, y_text + baseline - 10),
89                (0, 0, 255),
90                cv2.FILLED,
91            )
92            cv2.putText(
93                frame,
94                label,
95                (x1, y_text - 7),
96                cv2.FONT_HERSHEY_SIMPLEX,
97                0.5,
98                (0, 0, 0),
99                1,
100            )

```

Tracking Code

```
6  ✓ class ObjectTracker:
7  ✓     def __init__(self, tracker_type: str = "CSRT"):
8         self.tracker_type = tracker_type.upper()
9         self.tracker = None
10        self.is_initialized = False
11        self.last_bbox_xywh: Optional[Tuple[int, int, int, int]] = None
12
13  ✓     def _create_tracker(self):
14         if self.tracker_type == "CSRT":
15             return cv2.TrackerCSRT_create()
16         elif self.tracker_type == "KCF":
17             return cv2.TrackerKCF_create()
18         elif self.tracker_type == "MIL":
19             return cv2.TrackerMIL_create()
20         else:
21             raise ValueError(f"Unsupported tracker type: {self.tracker_type}")
22
23  ✓     def init(self, frame, bbox_xywh: Tuple[int, int, int, int]):
24         """
25         Initialize tracker using bbox from detection.
26         bbox must be (x, y, w, h)
27         """
28         x, y, w, h = [int(v) for v in bbox_xywh]
29
30         # Reject invalid boxes
31         if w <= 0 or h <= 0:
32             self.is_initialized = False
33             self.last_bbox_xywh = None
34             return False
35
36         self.tracker = self._create_tracker()
37
38         # Important:
39         # In some OpenCV versions tracker.init() returns None,
40         # so do not trust its return value.
41         self.tracker.init(frame, (x, y, w, h))
42         self.is_initialized = True
43         self.last_bbox_xywh = (x, y, w, h)
44         return True
45
```

```

46  ✓    def update(self, frame):
47        if self.tracker is None or not self.is_initialized:
48            return False, None
49
50        success, bbox = self.tracker.update(frame)
51
52        if not success:
53            return False, None
54
55        x, y, w, h = [int(v) for v in bbox]
56        self.last_bbox_xywh = (x, y, w, h)
57        return True, self.last_bbox_xywh
58
59    def reinitialize(self, frame, bbox_xywh: Tuple[int, int, int, int]):
60        return self.init(frame, bbox_xywh)
61
62    @staticmethod
63  ✓    def draw_tracking(frame, bbox_xywh, color=(255, 0, 0), text="Tracking"):
64        x, y, w, h = [int(v) for v in bbox_xywh]
65        cv2.rectangle(frame, (x, y), (x + w, y + h), color, 2)
66        cv2.putText(
67            frame,
68            text,
69            (x, max(20, y - 10)),
70            cv2.FONT_HERSHEY_SIMPLEX,
71            0.7,
72            color,
73            2,
74        )

```

Hybrid Code(Merged Algorithm by using both tracking and detection code)

```
147     def vision_worker(  
148         drone_data: OrtakData,  
149         stop_event,  
150         model_path: str = "best.pt",  
151         source: str = "0",  
152         conf_threshold: float = 0.5,  
153         target_class: Optional[str] = None,  
154         redetect_interval: float = 2.0,  
155         iou_threshold: float = 0.3,  
156         max_miss: int = 2,  
157         max_bbox_size: int = 50,  
158         start_second: float = 0.0,  
159         detection_confirmation_frames: int = 3,  
160     ):  
161         """Hybrid detection+tracking worker for the vision thread."""  
162         print(f"[VISION] Starting hybrid vision worker (source={source}, model={model_path})")  
163  
164         cap = _open_video_source(source)  
165         if not cap.isOpened():  
166             print(f"[VISION] Error: could not open source {source}")  
167             return  
168  
169         fps = cap.get(cv2.CAP_PROP_FPS)  
170         if fps > 0 and start_second > 0:  
171             cap.set(cv2.CAP_PROP_POS_FRAMES, int(fps * start_second))  
172  
173         detector = YoloDetector(model_path, conf_threshold=conf_threshold)  
174         tracker = ObjectTracker("CSRT")  
175  
176         local_mode = "DETECTION"  
177         tracked_class_name = target_class  
178         last_redetect_time = 0.0  
179         last_confidence = 0.0  
180         miss_count = 0  
181         detection_counter = 0 # Kaç frame üst üste hedef tespit edildi  
182         detect_skip_counter = 0  
183         DETECT_FRAME_SKIP = 2 # DETECTION modunda her 3 frame'de bir YOLO çalıştır  
184
```



```

185     while not stop_event.is_set():
186         ret, frame = cap.read()
187         if not ret:
188             print("[VISION] End of stream or cannot read frame.")
189             break
190
191         current_time = time.perf_counter()
192         timestamp = time.time()
193
194         if local_mode == "DETECTION":
195             detect_skip_counter = (detect_skip_counter + 1) % DETECT_FRAME_SKIP
196             if detect_skip_counter != 0:
197                 drone_data.update_vision_data(False, frame=frame)
198                 time.sleep(0.001)
199                 continue
200             detections = detector.detect(frame, target_class_name=target_class)
201             detections = [d for d in detections if d.bbox_xywh[2] <= max_bbox_size and d.bbox_xywh[3] <= max_bbox_size]
202             if detections:
203                 best_det = max(detections, key=lambda d: d.confidence)
204                 detection_counter += 1
205                 if detection_counter >= detection_confirmation_frames:
206                     # Yeterli onay alındı, tracking'e geç
207                     init_ok = tracker.init(frame, best_det.bbox_xywh)
208                     if init_ok:
209                         tracked_class_name = best_det.class_name
210                         local_mode = "TRACKING"
211                         last_redetect_time = current_time
212                         miss_count = 0
213                         detection_counter = 0 # Reset counter
214                         x1 = best_det.bbox_xywh[0]
215                         y1 = best_det.bbox_xywh[1]
216                         x2 = x1 + best_det.bbox_xywh[2]
217                         y2 = y1 + best_det.bbox_xywh[3]
218                         cx = (x1 + x2) / 2
219                         cy = (y1 + y2) / 2
220                         laser_zone = is_in_laser_zone(cx, cy, frame.shape[1], frame.shape[0])
221                         last_confidence = best_det.confidence
222                         drone_data.update_vision_data(

```

```

221                 last_confidence = best_det.confidence
222                 drone_data.update_vision_data(
223                     True, x1, y1, x2, y2,
224                     last_confidence, timestamp,
225                     frame=frame, laser_zone=laser_zone,
226                 )
227                 drone_data.signal_target_acquired()
228             else:
229                 # Tracker init başarısız, counter'ı sıfırla
230                 detection_counter = 0
231                 drone_data.update_vision_data(False, frame=frame)
232             else:
233                 # Henüz yeterli onay yok, detection modunda kal
234                 drone_data.update_vision_data(False, frame=frame)
235             else:
236                 # Hedef tespit edilemedi, counter'ı sıfırla
237                 detection_counter = 0
238                 drone_data.update_vision_data(False, frame=frame)
239
240         elif local_mode == "TRACKING":
241             success, tracked_bbox = tracker.update(frame)
242             if success:
243                 x1 = tracked_bbox[0]
244                 y1 = tracked_bbox[1]
245                 x2 = x1 + tracked_bbox[2]
246                 y2 = y1 + tracked_bbox[3]
247                 cx = (x1 + x2) / 2
248                 cy = (y1 + y2) / 2
249                 laser_zone = is_in_laser_zone(cx, cy, frame.shape[1], frame.shape[0])
250                 drone_data.update_vision_data(
251                     True, x1, y1, x2, y2,
252                     last_confidence, timestamp,
253                     frame=frame, laser_zone=laser_zone,
254                 )
255

```

```

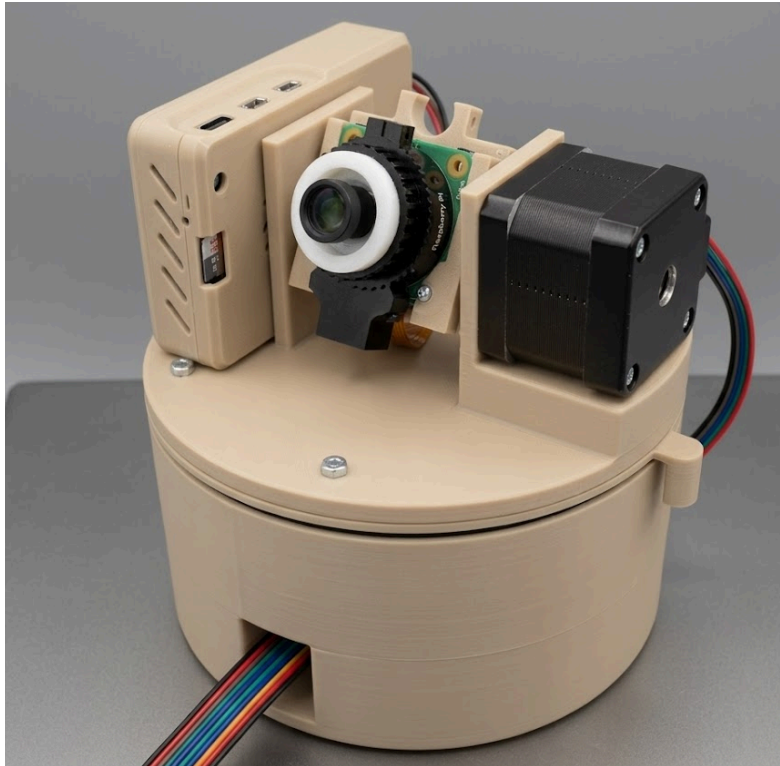
255
256         if (current_time - last_redetect_time) >= redetect_interval:
257             detections = detector.detect(frame, target_class_name=tracked_class_name)
258             detections = [d for d in detections if d.bbox_xywh[2] <= max_bbox_size and d.bbox_xywh[3] <= max_bbox_size]
259             matched_det = choose_matching_detection(detections, tracked_bbox, iou_threshold=iou_threshold)
260             if matched_det is not None:
261                 last_confidence = matched_det.confidence
262                 miss_count = 0
263                 tracker.reinitialize(frame, matched_det.bbox_xywh)
264                 last_redetect_time = current_time
265                 rx1 = matched_det.bbox_xywh[0]
266                 ry1 = matched_det.bbox_xywh[1]
267                 drone_data.set_yolo_recheck_bbox((rx1, ry1, rx1 + matched_det.bbox_xywh[2], ry1 + matched_det.bbox_xywh[3]))
268             else:
269                 miss_count += 1
270                 print(f"[VISION] YOLO re-check: no good match. Miss count: {miss_count}/{max_miss}")
271                 if miss_count >= max_miss:
272                     print("[VISION] Too many misses -> switching to DETECTION.")
273                     local_mode = "DETECTION"
274                     miss_count = 0
275                     detection_counter = 0 # Reset detection counter
276                     last_redetect_time = current_time
277                     drone_data.signal_target_lost()
278                     drone_data.update_vision_data(False, frame=frame)
279                     continue
280                 last_redetect_time = current_time
281         else:
282             print("[VISION] Tracking failed.")
283             local_mode = "DETECTION"
284             miss_count = 0
285             detection_counter = 0 # Reset detection counter
286             last_confidence = 0.0
287             drone_data.set_yolo_recheck_bbox(None)
288             drone_data.signal_target_lost()
289             drone_data.update_vision_data(False, frame=frame)
290

```

Appendix II

Long-Ranged Camera-Based Drone Detection System User Manual

This document provides basic information about the usage of the long-ranged camera-based drone detection system.



Electrical Wiring

The system's internal electrical wiring is robust. The only wiring the user must be concerned with are the two power lines which feed the mission computer and the electromechanical systems. The power line feeding the mission computer is quite straightforward. A 5V, 5A DC adapter is used to power the computer through a type-c USB port.

The power line leading to the electromechanical system consists of a 24V DC line and a ground. This may be provided through a DC power source as the current drawn by motors varies throughout the operation of the system. A current limit at 2A is recommended for the safety of the motor drivers. The system's ground connections are interconnected to form a common ground.

System Software

For starting the system, the user must run the 'top_module.py' script. This script starts the GUI and also initiates the threads which the system operates on. The first window to

pop-open is given in figure 1. The locked target will be encased in a bounding box which may be observed through the GUI.

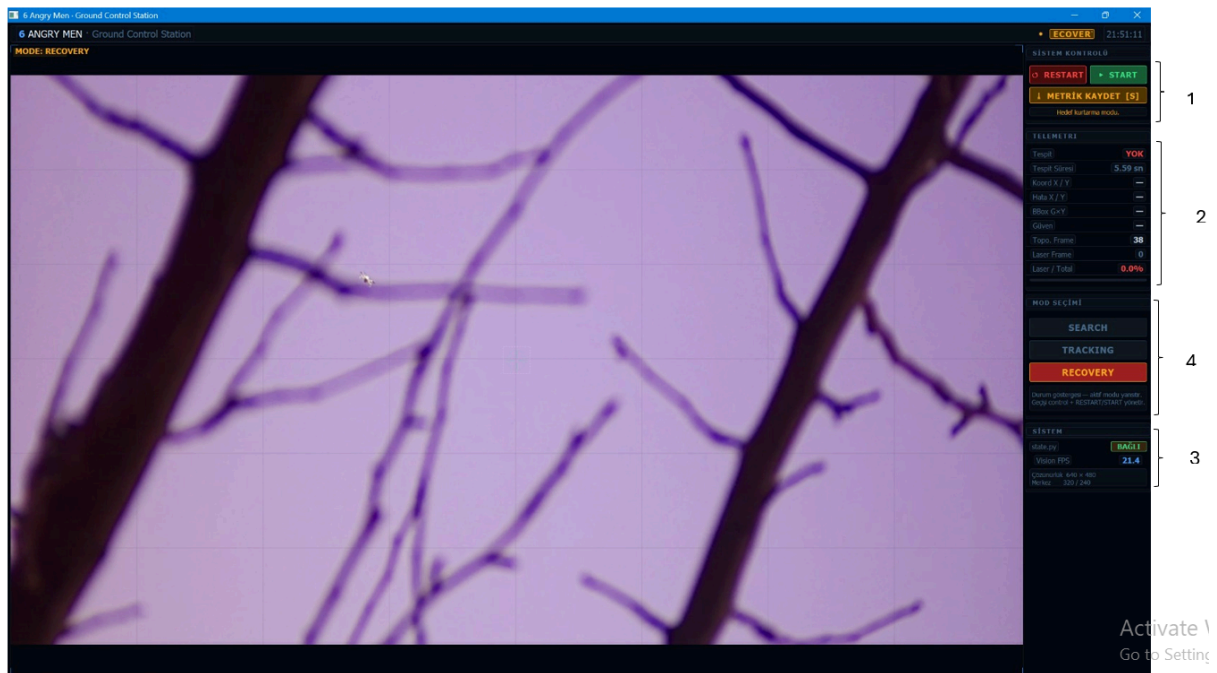


Figure-1

Figure-1 shows the GUI window with labels 1-4.

1. **System Control Bar:** The system control bar allows users to start, stop or restart the system. This allows for restarts and resets without having to physically cut power to the system during tuning or tests. 'Metrik Kaydet' button allows users to view the metrics of the system for specific tests and operations.
2. **Telemetry Bar:** The telemetry bar displays critical data for the viewing pleasure of the users. The existence of a lock, lock time, (x,y) coordinates, (x,y) error, bounding box dimensions, confidence data and other data are displayed at a consistent frequency.
3. **System Display Bar:** The system display bar displays non-critical data. As seen in figure-1, the data displayed includes the operation FPS, the display resolution, and the target coordinates in pixels.
4. **Operation Mode Bar:** The operation mode bar allows users to observe and choose the operation mode of the system.
 - a. In search mode, the system scans the designated region in the sky until a lock has been obtained.
 - b. The system enters tracking mode after a lock has been obtained and tries to keep the lock by tracking the target.
 - c. Recovery mode is activated when the lock on the target is lost. The system searches the vicinity of the last lock on the target. If the lock is renewed, the

system enters tracking mode again. If the lock could not be renewed in a designated amount of time, the system enters search mode.

The users may press 'P' on the keyboard to activate the PID tuner which is given in figure-2.



Figure-2

As seen in figure-2, many critical parameters may be modified and saved into the system through this window. It is important to note that this tuner may be used dynamically while the system is active and operating.

▲ Inputting the proportional, integral or differential variables haphazardly may result in catastrophic accidents and damage to the system!!

The 'Alpha' parameter is used for the exponential moving average calculation and determines the ratio of which value; the previous filtered output or the current raw input, will be used in the calculation of the current filtered output. The dead zone parameter determines a dead zone (in pixels) around the target where the system assumes convergence to target has been reached. The maximum steps parameter limits the number of steps that the motors could take for each movement command. This limit prevents extreme and erratic actuator movement.

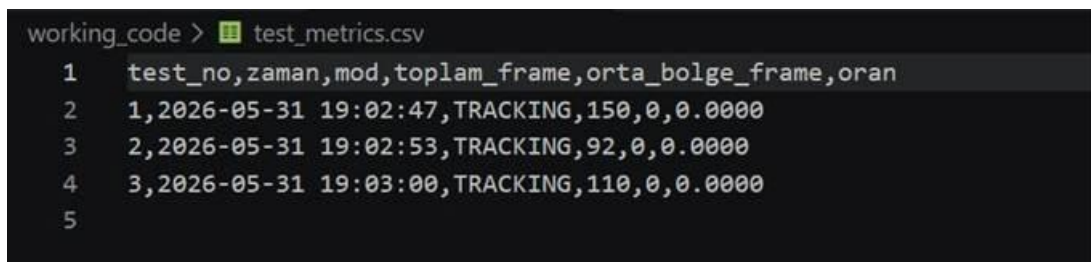


Figure-3

Figure-3 shows the terminal through which variables like test number, time, operation mode and total number of frames.